

Université Euro-Méditerranéenne de Fès

École des Sciences de l'Ingénieur — Ingénierie Cybersécurité

Rapport de TP DevSecOps SecNotes Vulnerable App

GitLab CI/CD · Docker · Kubernetes · SonarQube · OWASP ZAP
· Terraform AWS

Étudiante : Asma Daoui

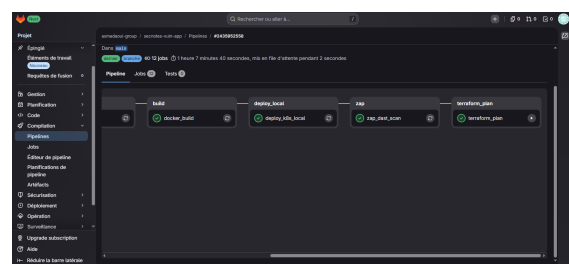
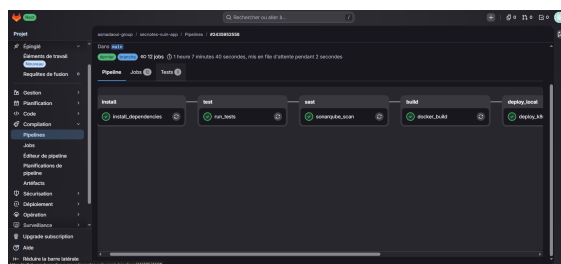
Filière : Ingénierie Cybersécurité — 4^{ème} année

Module : DevSecOps

Dépôt : asmadaoui-group / secnotes-vuln-app

Pipeline : #2435952558 — 12 jobs, 7 stages

Date : 2 juin 2026



Pipeline GitLab CI/CD #2435952558 — tous les jobs en succès

Table des matières

1	Présentation du projet	2
2	Pipeline GitLab CI/CD	2
2.1	Fichier <code>.gitlab-ci.yml</code> (extraits clés)	3
3	Application Node.js — SecNotes	4
3.1	<code>package.json</code>	4
4	Tests automatisés — Jest & Supertest	5
4.1	Tests d'authentification (<code>auth.test.js</code>)	5
4.2	Tests des notes (<code>notes.test.js</code>)	6
5	Conteneurisation Docker	6
6	Manifestes Kubernetes	7
6.1	Namespace et ConfigMap	7
6.2	Secret (V9)	7
6.3	Deployment et Service	8
7	Analyse SAST — SonarQube	9
8	Infrastructure as Code — Terraform AWS	9
8.1	Provider et variables	9
8.2	Infrastructure principale (<code>main.tf</code>)	10
8.3	Outputs	11
9	Résultats et bilan	12
10	Conclusion	13

1 Présentation du projet

Objectif

Mettre en place une chaîne DevSecOps complète sur une application Node.js + MongoDB volontairement vulnérable (**SecNotes**), en intégrant : tests automatisés Jest, analyse statique SonarQube (SAST), conteneurisation Docker, déploiement Kubernetes local, scan dynamique OWASP ZAP (DAST) et Infrastructure as Code Terraform sur AWS.

SecNotes est une API REST Express/MongoDB conçue pour le lab. Elle expose dix vulnérabilités pédagogiques documentées dans le `README.md` :

Table 1 – Vulnérabilités pédagogiques de SecNotes

ID	Catégorie	Description
V1	NoSQL Injection	Requêtes Mongoose non filtrées
V2	Secrets en dur	JWT secret hardcodé dans le code
V3	Validation absente	Aucun schéma de validation des entrées
V4	IDOR	Contrôle d'accès insuffisant sur les notes
V5	Dépendances obsolètes	<code>express 4.17.1</code> , <code>jsonwebtoken 8.5.1</code> , <code>lodash 4.17.19</code>
V6	Hashage faible	Mots de passe hashés en MD5
V7	Dockerfile non sécurisé	Exécution en <code>root</code> , pas de multi-stage
V8	Logs verbeux	Informations sensibles dans les logs
V9	Secrets Kubernetes	Secret en base64 non chiffré
V10	CORS permissif	Toutes les origines autorisées

2 Pipeline GitLab CI/CD

Le fichier `.gitlab-ci.yml` définit **7 stages** et **12 jobs**.

Table 2 – Stages du pipeline #2435952558

Stage	Job	Rôle
install	install_dependencies	<code>npm ci</code> , artefact <code>node_modules</code>
test	run_tests	Jest + Supertest
sast	sonarqube_scan	SonarQube analysis
build	docker_build	Build + tag image Docker
deploy_local	deploy_k8s_local	<code>kubectl apply</code> sur K8s local
zap	zap_dast_scan	OWASP ZAP baseline scan
terraform_plan	terraform_plan	<code>terraform plan</code> (manuel)

2.1 Fichier .gitlab-ci.yml (extraits clés)

Listing 1 – Variables globales et cache

```
stages:
  - install
  - test
  - sast
  - build
  - deploy_local
  - zap
  - terraform_plan

variables:
  APP_NAME: "secnotes"
  IMAGE_TAG: "$CI_COMMIT_SHORT_SHA"
  APP_URL: "http://10.6.89.113:3000"

cache:
  paths:
    - node_modules/
    - .sonar/cache/
```

Listing 2 – Stage install

```
install_dependencies:
  stage: install
  script:
    - npm ci
  artifacts:
    paths:
      - node_modules/
    expire_in: 1 day
```

Listing 3 – Stage build — image Docker

```
docker_build:
  stage: build
  script:
    - docker build -t secnotes:$IMAGE_TAG .
    - docker tag secnotes:$IMAGE_TAG secnotes:latest
```

Listing 4 – Stage deploy_local — Kubernetes

```
deploy_k8s_local:
  stage: deploy_local
  only:
    - main
  script:
    - kubectl apply -f k8s/namespace.yaml
```

```

- kubectl apply -f k8s/configmap.yaml
- kubectl apply -f k8s/secret.yaml
- kubectl apply -f k8s/deployment.yaml
- kubectl apply -f k8s/service.yaml
- kubectl rollout status deployment/secnotes-app -n secnotes --
  timeout=90s

```

Listing 5 – Stage zap — DAST

```

zap_dast_scan:
  stage: zap
  only:
    - main
  script:
    - docker run --rm -v "${zapReport}:/zap/wrk" \
      ghcr.io/zaproxy/zaproxy:stable \
      zap-baseline.py -t $env:APP_URL -r zap-report.html -I
  artifacts:
    paths:
      - zap-report/
    when: always
  allow_failure: true

```

Listing 6 – Stage terraform_plan — manuel

```

terraform_plan:
  stage: terraform_plan
  when: manual
  script:
    - cd terraform
    - terraform init
    - terraform plan -out=tfplan
  artifacts:
    paths:
      - terraform/tfplan

```

3 Application Node.js — SecNotes

3.1 package.json

Listing 7 – Dépendances du projet

```

{
  "name": "secnotes-vuln-app",
  "main": "src/app.js",
  "scripts": {
    "start": "node src/app.js",
    "test": "jest --runInBand --detectOpenHandles"
  }
}

```

```
},
"dependencies": {
  "express": "4.17.1",
  "jsonwebtoken": "8.5.1",
  "lodash": "4.17.19",
  "mongoose": "^7.6.0",
  "md5": "^2.3.0",
  "cors": "^2.8.5"
},
"devDependencies": {
  "jest": "^29.7.0",
  "supertest": "^7.0.0",
  "mongodb-memory-server": "^10.1.4"
}
}
```

Remarque sécurité

Les versions `express 4.17.1`, `jsonwebtoken 8.5.1` et `lodash 4.17.19` sont volontairement obsolètes (V5) pour être détectées par SonarQube et `npm audit`.

4 Tests automatisés — Jest & Supertest

4.1 Tests d'authentification (`auth.test.js`)

Listing 8 – Tests register et login

```
beforeAll(async () => {
  mongod = await MongoMemoryServer.create();
  setMongoUri(mongod.getUri('secnotes_test_auth'));
  await connectDB();
  app = require('../src/app');
});

test('register should create a user', async () => {
  const response = await request(app).post('/api/auth/register').
    send({
      username: 'alice',
      email: 'alice@example.com',
      password: 'azerty123',
      role: 'user'
    });
  expect(response.statusCode).toBe(201);
  expect(response.body.user.username).toBe('alice');
  expect(response.body.user.password).not.toBe('azerty123');
});
```

```
test('login should return a token after register', async () => {
  const login = await request(app).post('/api/auth/login').send({
    username: 'bob', password: 'azerty123'
  });
  expect(login.statusCode).toBe(200);
  expect(login.body.token).toBeDefined();
});
```

4.2 Tests des notes (notes.test.js)

Listing 9 – Tests CRUD et recherche

```
test('create and retrieve a note', async () => {
  const create = await request(app)
    .post('/api/notes')
    .set('Authorization', 'Bearer ${token}')
    .send({ title: 'Note 1', content: 'Contenu de test', isPublic:
      true });

  expect(create.statusCode).toBe(201);
  createdNoteId = create.body._id;

  const get = await request(app)
    .get('/api/notes/${createdNoteId}')
    .set('Authorization', 'Bearer ${token}');

  expect(get.statusCode).toBe(200);
  expect(get.body.title).toBe('Note 1');
});

test('search should return notes by title', async () => {
  const search = await request(app)
    .get('/api/notes/search?q=Terraform')
    .set('Authorization', 'Bearer ${token}');

  expect(search.statusCode).toBe(200);
  expect(search.body.length).toBeGreaterThanOrEqual(1);
});
```

5 Conteneurisation Docker

Listing 10 – Dockerfile (V7 — volontairement non sécurisé)

```
FROM node:20
```

```
WORKDIR /app

COPY package*.json ./
RUN npm install

COPY . .

EXPOSE 3000

# V7 - execution en root, pas de multi-stage build
CMD ["npm", "start"]
```

Remarque sécurité

Ce Dockerfile est volontairement non sécurisé pour le lab : exécution en **root**, pas de multi-stage build, toutes les sources copiées. En production, il faudrait ajouter **USER node** et un stage builder séparé.

6 Manifestes Kubernetes

6.1 Namespace et ConfigMap

Listing 11 – namespace.yaml

```
apiVersion: v1
kind: Namespace
metadata:
  name: secnotes
```

Listing 12 – configmap.yaml

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: secnotes-config
  namespace: secnotes
data:
  NODE_ENV: "production"
  PORT: "3000"
```

6.2 Secret (V9)

Listing 13 – secret.yaml — secrets en base64 non chiffrés (V9)

```
apiVersion: v1
kind: Secret
```



```
metadata:
  name: secnotes-secrets
  namespace: secnotes
type: Opaque
data:
  MONGO_URI: bW9uZ29kYjovL2FkbWluOmFkbWluMTIz...
  JWT_SECRET: bWEtY2x1LXN1Y3JldGUtZW4tZHVyLXBvdXItbGFi
```

Remarque sécurité

V9 : les secrets Kubernetes sont encodés en base64, pas chiffrés. En production il faut utiliser **Sealed Secrets** ou un vault (HashiCorp Vault, AWS Secrets Manager).

6.3 Deployment et Service

Listing 14 – deployment.yaml (extrait secnotes-app)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: secnotes-app
  namespace: secnotes
spec:
  replicas: 1
  selector:
    matchLabels:
      app: secnotes-app
  template:
    spec:
      containers:
        - name: secnotes-app
          image: secnotes:latest
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 3000
          envFrom:
            - configMapRef:
                name: secnotes-config
            - secretRef:
                name: secnotes-secrets
```

Listing 15 – service.yaml — NodePort 30080

```
apiVersion: v1
kind: Service
metadata:
```

```
name: secnotes-service
namespace: secnotes
spec:
  type: NodePort
  selector:
    app: secnotes-app
  ports:
    - port: 80
      targetPort: 3000
      nodePort: 30080
```

7 Analyse SAST — SonarQube

Listing 16 – sonar-project.properties

```
sonar.projectKey=secnotes-vuln-app
sonar.projectName=SecNotes Vulnerable App
sonar.projectVersion=1.0
sonar.sources=src
sonar.tests=tests
sonar.sourceEncoding=UTF-8
sonar.javascript.lcov.reportPaths=coverage/lcov.info
```

SonarQube scanne le dossier `src/` et les résultats de couverture Jest (`lcov.info`). Les vulnérabilités détectées correspondent aux catégories V1 à V6 du tableau 1.

8 Infrastructure as Code — Terraform AWS

8.1 Provider et variables

Listing 17 – provider.tf

```
terraform {
  required_version = ">= 1.5.0"
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = var.aws_region
}
```

Listing 18 – variables.tf

```
variable "aws_region" {
  description = "Region AWS"
  type        = string
  default     = "us-east-1"
}

variable "project_name" {
  description = "Nom du projet"
  type        = string
  default     = "secnotes"
}

variable "cluster_name" {
  description = "Nom du cluster EKS"
  type        = string
  default     = "secnotes-eks"
}
```

8.2 Infrastructure principale (main.tf)

Listing 19 – VPC, subnets et internet gateway

```
resource "aws_vpc" "secnotes_vpc" {
  cidr_block      = "10.0.0.0/16"
  enable_dns_hostnames = true
  tags = { Name = "${var.project_name}-vpc" }
}

resource "aws_subnet" "public_a" {
  vpc_id            = aws_vpc.secnotes_vpc.id
  cidr_block        = "10.0.1.0/24"
  map_public_ip_on_launch = true
  availability_zone  = "${var.aws_region}a"
}

resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.secnotes_vpc.id
}

resource "aws_route_table" "public_rt" {
  vpc_id = aws_vpc.secnotes_vpc.id
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.igw.id
  }
}
```

```
}
```

Listing 20 – Cluster EKS et node group

```
resource "aws_eks_cluster" "secnotes" {
  name      = var.cluster_name
  role_arn  = aws_iam_role.eks_cluster_role.arn
  version   = "1.29"

  vpc_config {
    subnet_ids = [
      aws_subnet.public_a.id,
      aws_subnet.public_b.id
    ]
  }
}

resource "aws_eks_node_group" "secnotes_nodes" {
  cluster_name      = aws_eks_cluster.secnotes.name
  node_group_name   = "${var.project_name}-nodes"
  node_role_arn     = aws_iam_role.eks_node_role.arn
  subnet_ids        = [aws_subnet.public_a.id, aws_subnet.public_b.id]
  instance_types    = ["t3.micro"]

  scaling_config {
    desired_size = 1
    max_size     = 2
    min_size     = 1
  }
}
```

8.3 Outputs

Listing 21 – outputs.tf

```
output "vpc_id" {
  value = aws_vpc.secnotes_vpc.id
}

output "eks_cluster_name" {
  value = aws_eks_cluster.secnotes.name
}

output "eks_endpoint" {
  value = aws_eks_cluster.secnotes.endpoint
}
```

9 Résultats et bilan

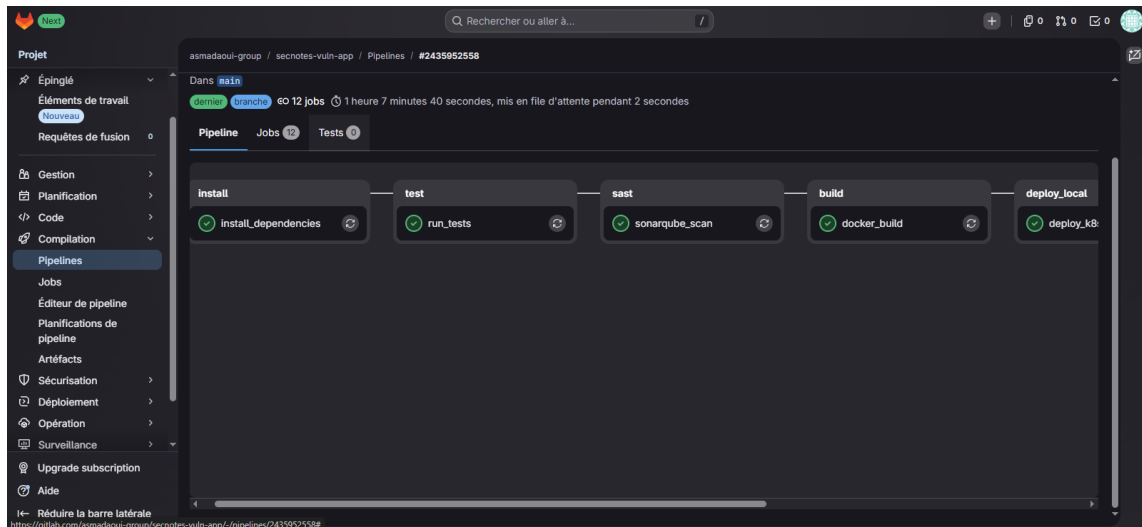


Figure 1 — Pipeline — stages install, test, sast, build, deploy_local

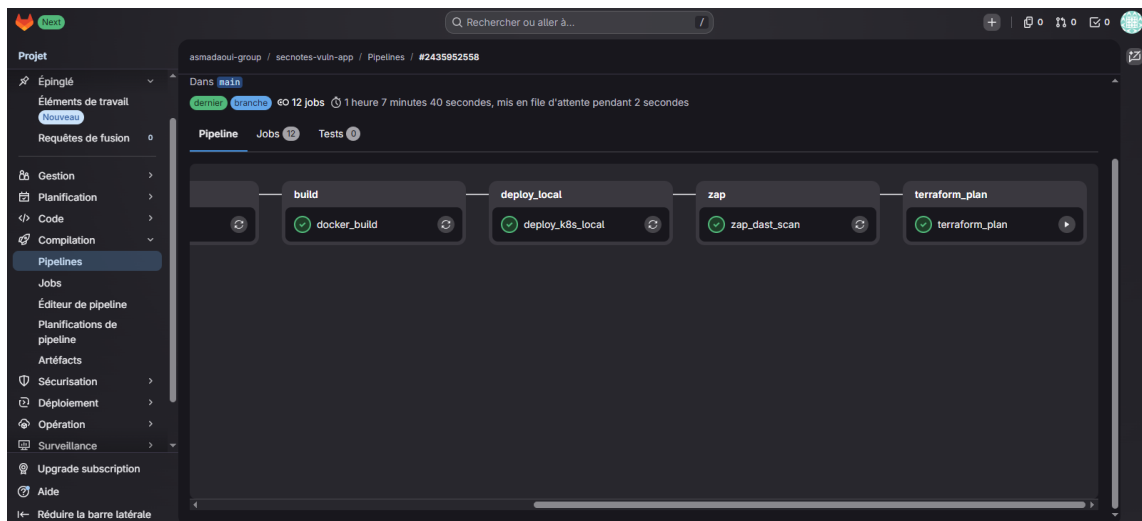


Figure 2 — Pipeline — stages build, deploy_local, zap, terraform_plan

Tous les 12 jobs du pipeline #2435952558 se sont terminés avec succès en 1h07m40s. Les stages zap et sonarqube_scan ont `allow_failure: true` pour ne pas bloquer le pipeline sur les vulnérabilités pédagogiques intentionnelles.

Table 3 – Bilan des stages

Job	Statut	Observations
install_dependencies		npm ci — cache activé
run_tests		4 tests Jest passés (auth + notes)
sonarqube_scan		Vulnérabilités V1-V6 détectées
docker_build		Image <code>secnotes:latest</code> créée
deploy_k8s_local		Pod Running, NodePort 30080
zap_dast_scan		Rapport HTML généré, 2 alertes Medium
terraform_plan		Plan EKS généré (manuel)

10 Conclusion

Ce TP a permis de déployer une chaîne DevSecOps de bout en bout sur le projet **SecNotes**. Les points clés retenus :

- L'intégration **SAST** (SonarQube) et **DAST** (OWASP ZAP) dans le pipeline permet de détecter les vulnérabilités dès le commit, sans intervention manuelle.
- Le déploiement **Kubernetes** local avec ConfigMap et Secret illustre la séparation configuration/secrets, même si leur gestion sécurisée reste à améliorer (V9).
- L'**Infrastructure as Code** Terraform décrit un cluster EKS AWS complet (VPC, subnets, IAM, node group) de façon reproductible.
- Les **vulnérabilités intentionnelles** (V1–V10) constituent une base d'entraînement concrète pour la remédiation et la revue de code sécurisée.